

Università della Calabria

Dipartimento di Matematica e Informatica



Corso di Laurea Magistrale in Informatica

Tesi di Laurea

PABEL: Plugin ABE for LLM

**Ensuring Document Confidentiality for AI Agents via
Attribute-Based Encryption**

Relatori	Candidato
Ch.mo Prof. Giovambattista Ianni	Claudio Lucisano
Dott. Ing. Andrea Altarui	Matricola 231871

Anno Accademico 2025/2026

Indice

1	Introduction	3
1.1	Context and Motivations	3
1.2	Activities done and Results	3
1.3	Technologies and Tools	4
1.4	Thesis outline	5
2	Motivation and Threat Model	6
2.1	Problem Statement	6
2.2	Actors and Goals	6
2.3	Assumptions	6
2.4	Adversary Model	6
2.5	Security Goals	6
3	Background	7
3.1	Attribute-Based Encryption	7
3.1.1	From Identity-Based to Attribute-Based Encryption	7
3.1.2	Ciphertext-Policy versus Key-Policy ABE	7
3.1.3	Multi-authority ABE	7
3.1.4	Functional Encryption as a Generalization	7
3.2	Identity and Access Management	7
3.2.1	OAuth 2.0 and PKCE	7
3.2.2	Keycloak	7
3.3	The Model Context Protocol	7
3.4	Hook-Based Interception in AI Coding Agents	7
4	Related Work	8
5	System Design	9
6	Security Analysis	10
7	Implementation	11
7.1	File Viewer	11

7.1.1	File Structure	11
7.1.2	File Viewer	13
8	Evaluation	14
9	Conclusions	15
10	Bibliography	16

1 Introduction

1.1 Context and Motivations

The constant and rapid evolution in the field of Artificial Intelligence has result in a day-to-day use of AI agents, thus the need of security and privacy when dealing with tools that, potentially, could harvest personal or company data for a variety of reasons. More in detail the concept that we were trying to focus on, was the scenario in which an employee, by chance or intentionally, had its AI agent to look at a confidential file which the company doesn't want to be leaked or simply does not trust the AI agent's vendor.

This particular fear, to not trust AI vendors, it's not unfounded; several recent incidents show how sensible data can end up outside the boundaries an organization intended to enforce, or how AI providers themselves may proceed to process user data in ways that raise legitimate concerns.

A first well known example is the 2023 Samsung incident, where some employees pasted proprietary source code into a public LLM chatbot for debugging purposes, effectively transmitting confidential IP (intellectual property) to an external, unintended infrastructure. This incident led Samsung to ban all AI tools altogether[1]

A second, more institutional, example involves OpenAI: in december 2024, the Italian Data Protection Authority fined OpenAI for €15 million, finding that ChatGPT was processing users' personal data to train underlying models without the legal basis required by the GDPR; not to mention a security breach, that took place in march 2023, which exposed users' data and payment information[2].

Also Anthropic was acting in regard of the guarantees that have to be assumed by a vendor, in particular it had to update its customer terms[3]. They explicitly stated that each conversation made with Claude could be used for model training, with the exception of Claude for Work, Claude Enterprise, Claude Gov and any API-based access[3].

1.2 Activities done and Results

In this thesis, we have designed and implemented a plugin for AI agents that allows to decrypt files using Attribute-Based Encryption (ABE) in order to ensure that only authorized users and authorized AI Agents can access specific parts of the file. The plugin is designed to be easily integrated into existing AI agents, providing a seamless way to

enhance security and privacy in AI-driven environments. Such security enhancement is achieved by encrypting files with ABE, which allows for fine-grained access control based on user and AI agent attributes. The plugin intercepts file access requests from the AI agent, authenticates the user and checks, via server, if both the user's attributes and the agent's attributes match the encryption policy; thus the server decrypts the specific parts of the file that both the user and *that* agent are authorized to see and returns it to the agent. This approach ensures that sensitive information remains protected when accessed by AI agents.

1.3 Technologies and Tools

To achieve the goals of this thesis, several technologies and tools have been utilized. The core of the security mechanism is based on Attribute-Based Encryption (ABE), more in particular we have used the open source library **OpenABE**[4], which we had to upgrade to a more recent version by updating its dependencies and fixing some bugs that were present in the original code.

Another important component of the system is the authentication and authorization mechanism, which is implemented using **Keycloak**, an open-source identity and access management solution. Keycloak provides a robust framework for managing user identities, roles, and permissions, which is essential for enforcing access control policies in the context of ABE.

The plugin is designed to be compatible with AI agents that support plugin integration, and it has been tested with specific AI agents to ensure proper functionality. The AI agents used for testing are:

- **Claude Code**, an AI agent developed by Anthropic;
- **Codex**, an AI agent developed by OpenAI;
- **Copilot for VSCode**, an AI agent developed by Microsoft, integrated into the Visual Studio Code IDE.

The server-side component of the system is implemented in Python and it was containerized using **Rancher Desktop**, a tool that allows to run Kubernetes clusters on a local machine. This setup enables easy deployment and management of the server component, ensuring scalability and reliability in handling decryption requests from AI agents.

Alongside the server container, there were deployed two other containers: one for the Keycloak server and another for a **PostgreSQL** database, which is used by Keycloak to store user and role information. This containerized approach simplifies the deployment process and allows for easy scaling of the system as needed.

1.4 Thesis outline

2 Motivation and Threat Model

2.1 Problem Statement

The main problem that this thesis aims to solve comes from a consideration while thinking how AI can be implemented in a blue team scenario. Even if this topic could be an argument of discussion by itself, many perplexities arise when we think how data is exchanged considering AI agents actively inside the attack surface.

In particular, the problem is that AI agents are not able to distinguish between sensitive and non-sensitive data, and they are not able to understand the context of the data they are processing. This means that if an AI agent is given access to sensitive data, it could potentially leak that data to an attacker, either intentionally or unintentionally. Of course, it is imperative to also consider the intrinsic vulnerability of AI agents to prompt injection attacks, which could be just another mean to the exfiltration of sensitive data as upmentioned.

One method to mitigate this problem would be to utilize cryptography, but that alone comes with its own set of challenges. More in detail, both an AI agent and a user cannot benefit from data that is encrypted with a symmetric key: it is easy to see how both of them would require a decryption of said data, otherwise they would not be able even to understand it, leave alone to use it.

One possible alternative could be Homomorphic Encryption and, surely, it is a very promising topic, but as for today its computational limitations make it not suitable for a real-world scenario, especially if we consider the amount of data that an AI agent should process.

2.2 Actors and Goals

2.3 Assumptions

2.4 Adversary Model

2.5 Security Goals

3 Background

3.1 Attribute-Based Encryption

3.1.1 From Identity-Based to Attribute-Based Encryption

3.1.2 Ciphertext-Policy versus Key-Policy ABE

3.1.3 Multi-authority ABE

3.1.4 Functional Encryption as a Generalization

3.2 Identity and Access Management

3.2.1 OAuth 2.0 and PKCE

3.2.2 Keycloak

3.3 The Model Context Protocol

3.4 Hook-Based Interception in AI Coding Agents

4 Related Work

5 System Design

6 Security Analysis

7 Implementation

7.1 File Viewer

7.1.1 File Structure

To start approaching the concept, it is imperative to subdivide the problem into smaller ones. The first problem that came to mind was to find a way to have into a single file multiple parts, each with its own encryption. To solve this problem, we did create a JSON structure that could contain each encrypted section of the file, i.e. an array containing a single object per section that has two properties: the policy requested to decipher the ciphertext and the ciphertext itself.

Listing 1: .abe format v1

```
{
  "format": "abe-doc",
  "version": 1,
  "sections": [
    {
      "policy": "livello = x",
      "ciphertext": CIPHERTEXT_Lx
    },
    {
      "policy": "livello = y",
      "ciphertext": CIPHERTEXT_Ly
    },
    {
      "policy": "livello = z",
      "ciphertext": CIPHERTEXT_Lz
    }
  ]
}
```

It's quite easy to point out that this implementation is very inefficient when the number of sections grows. To optimize the amount of operations needed and the structure of the document itself, it was improved in the version 2 of the structure:

Listing 2: .abe v2

```
{
```

```

"format": "abe-doc",
"version": 1,
"sections": [
  {
    "group": 0
  },
  {
    "group": 1
  },
  {
    "group": 2
  },
  {
    "group": 1
  },
  {
    "group": 0
  }
],
"groups": [
  {
    "policy": "livello >= 3",
    "ciphertext": CIPHERTEXT_L3
  },
  {
    "policy": "livello >= 2",
    "ciphertext": CIPHERTEXT_L2
  },
  {
    "policy": "livello >= 1",
    "ciphertext": CIPHERTEXT_L1
  }
]
}

```

This structure provides one key feature: the ability to call the decrypt function n times, where n is the number of unique attributes used to rules the sections. This is a more sustainable approach than v1, since, in that case, the decrypt function would be called

exactly m times, where m is the amount of each section in the document. It's a fair assumption to say that m , *usually*, is greater than n .

7.1.2 File Viewer

Having a structured document by itself is useless if there is not a way to read and edit it, so it was necessary to create such application. The file viewer operates as a simple file editor, with the ability to encrypt and decrypt a file using a user's private key. This private key, as seen before, has to be created and stored beforehand. A user could also authenticate itself via **Keycloak** login and so a token with its attribute will be elaborated and used in substitution of the user's key.

8 Evaluation

9 Conclusions

Conclusions

10 Bibliography

Riferimenti bibliografici

- [1] Siladitya Ray. *Samsung Bans ChatGPT And Other Chatbots For Employees After Sensitive Code Leak*. <https://www.forbes.com/sites/siladityaray/2023/05/02/samsung-bans-chatgpt-and-other-chatbots-for-employees-after-sensitive-code-leak/>. Forbes; ultimo accesso 5 agosto 2026. 2023.
- [2] Garante per la protezione dei dati personali. *ChatGPT, il Garante privacy chiude l'istruttoria. OpenAI dovrà realizzare una campagna informativa di sei mesi e pagare una sanzione di 15 milioni di euro*. <https://www.garanteprivacy.it/home/docweb/-/docweb-display/docweb/10085432>. Provvedimento n. 755 del 2 novembre 2024; ultimo accesso 5 agosto 2026. 2024.
- [3] Anthropic. *Updates to Consumer Terms and Privacy Policy*. <https://www.anthropic.com/news/updates-to-our-consumer-terms>. Ultimo accesso 5 agosto 2026. 2025.
- [4] Zeutro. *OpenABE: An Open Source cryptographic library, with implementation of Attribute-Based Encryption in C/C++*. <https://github.com/zeutro/openabe>. 2020.